Studio Nx

JS 01 — refreshAccessToken()

2026-10-07

JS 01 — `refreshAccessToken()`

항목	내용
파일	`studionx/frontend/static/js/core/api.js`
작업	P1-02 프론트 공통 JS · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
다음	02 `apiFetch()`

1. 하는 일

리프레시 쿠키로 Access Token 을 다시 받는다. 성공하면 `true`, 실패하면 `false`.

2. 언제 쓰나

Access Token 은 메모리에만 있어서 **페이지를 옮기면 사라지고**, **30분이 지나면 만료**된다. 그때 로그인을 다시 하지 않고 토큰만 새로 받으려고 쓴다.

상황	부르는 곳
페이지를 열고 첫 API 요청 (토큰이 아직 없음)	`apiFetch()` — 요청 전에 먼저 부름
요청했는데 401 (토큰 만료)	`apiFetch()` — 재발급 후 같은 요청을 한 번 더
로그인이 필요한 페이지에 들어올 때	`isLoggedIn()` → `requireLogin()` (마이페이지 · 내 프로젝트 · 회원 정보 · 시퀀서)
헤더를 로그인 상태로 바꿀 때	`isLoggedIn()` (모든 페이지 헤더)

페이지 코드가 직접 부르는 일은 거의 없다 — 위 함수들을 거친다.

리프레시 쿠키 (`snx_refresh`)

로그인을 오래 유지하려고 서버가 브라우저에 심는 쿠키. 안에 리프레시 토큰이 들어 있다. (서버 쪽은 P1-05 `api/v1/auth.py`)

언제	무슨 일
로그인할 때 (`POST /auth/login`)	서버가 쿠키를 심는다. **자동 로그인 체크** → 3일 유지 (브라우저를 닫아도 남음) · **체크 안 함** → 세션 쿠키 (브라우저를 닫으면 사라짐)
재발급할 때 (`POST /auth/refresh`, 이 함수)	브라우저가 쿠키를 자동으로 보냄 → 서버가 확인하고 새 Access Token 을 줌
로그아웃할 때 (`POST /auth/logout`)	서버가 쿠키를 지운다
그 밖의 API 요청	쿠키가 **실리지 않음** — `path=/api/v1/auth` 로 좁혀 둠

속성	뜻
`HttpOnly`	JS 에서 읽을 수 없음 → 스크립트 공격(XSS)으로 훔칠 수 없음

속성	뜻
`SameSite=Lax`	다른 사이트가 보낸 POST 에는 안 실림 → CSRF 대비
`Secure` (배포)	HTTPS 에서만 전송
`path=/api/v1/auth`	로그인 · 재발급 · 로그아웃 요청에만 실림

토큰	어디에	수명	쓰임
Access Token	메모리 (`api.js` 변수)	30분	모든 API 요청의 `Authorization: Bearer …`
리프레시 토큰	HttpOnly 쿠키	3일 또는 브라우저를 닫을 때까지	Access Token 을 다시 받을 때만

쿠키가 없거나 만료되면 `/auth/refresh` 가 401 → 이 함수가 `false` → 로그인 안 된 상태로 처리.

3. 코드

import (파일 맨 위) `api.js` 는 다른 파일을 import 하지 않는다 (공통 모듈의 맨 아래 단계). 이 함수가 쓰는 `refreshing · setAccessToken · clearAccessToken · API_BASE` 는 같은 파일 안에 있고, `fetch` 는 브라우저 기본 기능이다.

함수

```
export async function refreshAccessToken() {
  if (refreshing) return refreshing;
  refreshing = (async () => {
    try {
      const res = await fetch(`${API_BASE}/auth/refresh`, {
        method: 'POST',
        credentials: 'same-origin',
        headers: { Accept: 'application/json' },
      });
      const data = res.ok ? await res.json().catch(() => null) : null;
      if (!data?.access_token) {
        clearAccessToken();
        return false;
      }
      setAccessToken(data.access_token);
      return true;
    } catch {
      clearAccessToken();
      return false;
    } finally {
      refreshing = null;
    }
  })();
  return refreshing;
}
```

문법 풀이

import 문법	뜻
`export` (함수 앞)	다른 파일이 `import { 함수 }` 로 가져다 쓸 수 있게 내보냄. 없으면 이 파일 안에서만 쓸 수 있음

`if (!data?.access_token)` — "토큰을 못 받았으면"

부분	뜻
`data?.access_token`	**옵셔널 체이닝(`?.`)**: `data` 가 `null`·`undefined` 면 오류 없이 `undefined`, 아니면 `data.access_token`
`!`	값이 없거나(`undefined`·`null`) 빈 문자열이면 `true`

`data`	`data?.access_token`	`!data?.access_token`	결과
`null` (실패 응답 · JSON 아님)	`undefined`	`true`	실패 처리
`{}` (토큰 없음)	`undefined`	`true`	실패 처리
`{ access_token: '' }`	`''`	`true`	실패 처리
`{ access_token: 'eyJ…' }`	`'eyJ…'`	`false`	성공 처리

`?.` 없이 `data.access_token` 이라고 쓰면, `data` 가 `null` 일 때 `TypeError: Cannot read properties of null` 로 멈춘다. 같은 뜻을 길게 쓰면 `if (data === null || data === undefined || !data.access_token)`.

`res.ok ? await res.json().catch(() => null) : null` — 성공 응답(200~299)일 때만 본문을 JSON 으로 읽고, 읽다 실패하면 `null`. 실패 응답이면 읽지 않고 `null`. 그래서 아래 `if` 하나로 모든 실패를 거른다.

4. 흐름

1. 재발급이 이미 진행 중이면 그 Promise 를 돌려준다.
2. `POST /api/v1/auth/refresh` — 쿠키는 브라우저가 자동으로 싣는다.
3. 응답에 `access_token` 이 있으면 저장 → `true`.
4. 401 · 네트워크 오류 · 토큰 없는 응답 → 토큰 지움 → `false`.
5. 끝나면 `refreshing` 을 비운다.

5. 왜 이렇게

선택	이유
진행 중인 Promise 공유 (`refreshing`)	여러 요청이 동시에 401 이어도 재발급은 한 번. 리프레시 토큰을 한 번 쓰고 바꾸는 방식에서도 안전
토큰을 JS 가 넣지 않음	리프레시 토큰은 HttpOnly 쿠키 — JS 에서 안 보임(XSS 대비). `credentials: 'same-origin'` 이면 자동 전송
실패해도 던지지 않음	부르는 쪽(`apiFetch` · `isLoggedIn`)은 성공 여부만 필요
`res.json().catch(() => null)`	JSON 이 아닌 응답도 예외 없이 실패로 처리
`finally` 에서 비움	성공 · 실패와 상관없이 다음 호출은 새로 요청

6. 시험 (fetch 를 가짜로 — `/auth/refresh` 는 P1-05 에서 구현)

경우	결과
200 + `access_token`	`true`, 토큰 저장, 주소 · POST · 쿠키 옵션 확인
401	`false`, 토큰 지움
네트워크 오류	`false`, 토큰 지움, 예외 없음
200 인데 토큰 없음	`false`

경우	결과
동시에 3번 호출	fetch 1번, 셋 다 `true`
끝난 뒤 다시 호출	새로 요청, 새 토큰